

Lab 4: Patches and a Tiny Contrastive Loss

Flatten image patches, then InfoNCE on four constructed pairs.

Work in a **Jupyter** notebook with **PyTorch** and `matplotlib`. No pretrained CLIP weights this week: the point is the geometry.

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np
```

1. Patches

1. Make a 32×32 RGB tensor (noise or a simple gradient is fine). Plot one channel.
2. Split it into 8×8 patches. How many tokens? Flatten each patch to a vector of length $8 \cdot 8 \cdot 3$.
3. Map patches with `nn.Linear(192, 16)` to a fake embedding width. Print the resulting shape (N, 16).

2. A tiny contrastive batch

Construct **four** image vectors and **four** text vectors in 2-D so that matches sit on a diagonal after cosine.

```
img = torch.tensor([[1.0, 0.0], [0.0, 1.0], [-1.0, 0.0], [0.0, -1.0]])
txt = img.clone()          # perfect matches
# add a little noise to txt[1] so the match is not trivial
txt[1] = torch.tensor([0.1, 0.9])
```

1. Cosine matrix $S = \hat{T}\hat{T}^T$ (L2-normalize rows first). Heatmap S .
2. Contrastive loss, image-to-text:

```
loss = F.cross_entropy(S / 0.07, torch.arange(4))
```

Print `loss`. Then replace `txt` with shuffled rows and print the new loss. It should rise.

3. Zero-shot toy: treat `txt` as class prompts. For `img[0]`, which row of `txt` is nearest? Did shuffling break it?

3. Write-up

Four sentences: what a patch token is, why positions would still be needed on a real image, what the diagonal of S means, and one way web-scale pairs are dirtier than this toy.